Customer Module Documentation

Overview

The Customer module provides an overview of the food delivery experience from the customer's side. The module is what users see, use and interact with. Users can browse restaurants, order food, choose their preferred paying method and track their order in real time.

Frontend Structure

Customers can:

Browse Restaurants

- Check all available restaurants in the area
- Filter restaurants by cuisine type
- Search for restaurants by name
- Filter restaurants by highest rated && quickest delivery time

Place an order

- Placing an order by choosing a menu item
- Adding menu items to cart

Cart view

- View the cart and its items
- Update the cart by removing and adding new items and items quanitty
- Clearing the cart
- Applying the promo code 'PROMO26'
- Proceeding to checkout

Choose a payment method

- The user can choose a preferred payment method:
  - by **cash** at order arrival
  - by **card** directly in the website by giving the card details

Track orders in Real-Time

- The user can see the order tracking **live** through:
  - **status updates** - get notified when the order is being prepared, ready, delivering and delivered
  - **chat with restaurant** - viewing the order updates, live, through the chat by the restaurant and messaging the restaurant
  - **delivery time estimates** - know when to expect the order

Review the Experience after Order Delivery

- The user can review the service provided by selecting a preferred review emoji and by writing a comment

Frontend Structure

```
client/src/modules/customer/
├── customer-home/ # Dashboard with navigation cards
├── restaurant-browsing/ # Restaurant list with filters
├── view-menu/ # Menu display by restaurant
├── cart-page/ # Shopping cart management
├── checkout-page/ # Order placement & payment
├── order-tracking/ # Real-time order status
├── feedback/ # Post-delivery feedback
├── paymentt-method/ # Payment method selection
├── customer.model.ts # TypeScript interfaces
├── customer.routes.ts # Module routing configuration
├── customer.service.ts # HTTP client for backend API
└── websocket.service.ts # Real-time order updates
```

Backend Structure

```
server/src/modules/customer/
├── customer.routes.ts # API endpoints for customer actions
├── customer.service.ts # Business logic - processing orders

server/src/domains/restaurant/
├── restaurant.routes.ts # Restaurant data logic
```

Key Data Models:

```
interface Restaurant {
restaurantId: number;
name: string;
cuisineType: string;
address: string;
rating: number;
deliveryTime: string;
isOpen: boolean;
minOrderAmount?: number;
}
```

```
interface MenuItem {
dishId: number;
name: string;
description: string;
```

```
price: number;
category: string;
imageUrl?: string;
isAvailable?: boolean;
}
```

```
interface CartItem {
dishId: number;
name: string;
price: number;
quantity: number;
restaurantId: number;
restaurantName: string;
}
```

Module Routes

Route	Component	Description
`/customer`	CustomerHomeComponent	Main dashboard
`/customer/home`	CustomerHomeComponent	Other path to dashboard
`/customer/restaurants`	RestaurantBrowsingComponent	Browsing all restaurants
`/customer/menu/:id`	ViewMenuComponent	View restaurant menu items
`/customer/cart`	CartPageComponent	Shopping cart
`/customer/checkout`	CheckoutComponent	Placing the order
`/customer/tracking/:id`	OrderTrackingComponent	Order id for real time order tracking
`/customer/feedback`	FeedbackComponent	Post-delivery feedback

Customer route component details

CustomerHomeComponent

- Display navigation cards for customer actions
- Implement order tracking dialog for users without active orders to be redirected to restaurant browsing
- LocalStorage gets automatically cleared up in order to not track old order tracking ids
- Angular signals for reactive state management

RestaurantBrowsingComponent

- Fetch and display all active restaurants
- Provides cuising type filtering (Austrian, Italian,American,Japanese)
- Implement search by restaurant name, cuisine or address

- Implements sorting by highest rated or fastest delivery time

ViewMenuComponent

- Loads restaurant details and menu via RestaurantBrowsingService
- Organizes dishes by categories with filter buttons
- Display dish images or emoji placeholders
- Enables adding preferred item to cart with live updates

CartPageComponent

- Implements CartService for cart state management
- Provides quantity updates controls
- Provides clear cart button
- Implements promo code application (PROMO26 with 5€ discount)
- Redirects the user to checkout page
- Enables cart validation before checkout

CheckoutComponent

- Implements PaymentMethodComponent for payment selection
- Supports credit card or cash on delivery payments
- Collects delivery address information (set by default)
- Creates actual order ids and stores them into localStorage for OrderTrackingComponent retrieval
- After a successfull order placement it redirects to order tracking component

OrderTrackingComponent

- Displays order details and current status
- Show estimated delivery time based on distance
- Order progress preview in 6 stages:
    1. **Placed** (at order placement)
    2. **Accepted**
    3. **Preparing**
    4. **Ready**
    5. **On the way**
    6. **Delivered**
- Calculates distance using grid coordinates

FeedbackComponent

- Allows customer to rate the service
- Collects feedback
- Submits feedback to backend

Services

CustomerService

- Main HTTP customer for customer-related API calls

- Methods used:
  - `getRestaurants()`: get all active restaurants
  - `getRestaurantById(id)`: get restaurant details
  - `getRestaurantMenu(id)`: get menu items
  - `getRestairantCategories(id)`: get menu categories
  - `getCartCount()`: get total cart items

RestaurantBrowsingService

- Delegated to CustomerService for data fetching
- `getRestaurantWithMenu()`: parallel data loading
- `Promise.all()`: API calls

CartService

- Manage cart state in localStorage
- Methods used:
  - `addToCart()`: add items with restaurant validation
  - `removeFromCart()`: remove item by dishId
  - `changeQuantity()`: update item quantity
  - `clearCart()`: empty cart
  - `getTotal()`: calculate final price with delivery fee

WebSocketService

- Simulates real- time Websocket connection for order tracking
- Emits order status updates through RxJS
- Calculates delivery time based on grid distance
- Simulates restaurant chat messaes
- Methods used:
  - `connect()`: initialize Websocket connection
  - `disconnect()`: disconnect Websocket connection
  - `sendMessage()`: send chat message
  - `calculateDistance()`: Calculating distance based on grid coordinates
  - `simulateStatusUpdates()`: simulates automatic status changes

Architrcture flow

```
flowchart TD
subgraph CustomerModule["Customer Module"]
CH[CustomerHomeComponent]
RB[RestaurantBrowsingComponent]
VM[ViewMenuComponent]
CP[CartPageComponent]
CO[CheckoutComponent]
OT[OrderTrackingComponent]
FB[FeedbackComponent]
end
```

```
    subgraph Services
        CS[CustomerService]
        RBS[RestaurantBrowsingService]
        CartS[CartService]
        WSS[WebSocketService]
    end

    subgraph Backend
        CR[customer.routes.ts]
        CServ[customer.service.ts]
        CRepo[customer.repository.ts]
    end

    CH --> CS
    RB --> RBS
    VM --> RBS
    VM --> CartS
    CP --> CartS
    CO --> CartS
    OT --> WSS
    FB --> CS

    RBS --> CS
    CS --> CR
    CR --> CServ
    CServ --> CRepo
```

Backend Implementation

API endpoints

- Provides endpoints for customer operations
- Routes require customer role via `requireRole` middleware
- Endpoints used:

Method	Endpoint	Description
GET	`/api/customer/restaurants`	Lists active restaurants
GET	`/api/customer/restaurants/:id`	Restaurant details
GET	`/api/customer/restaurants/:id/menu`	Menu items
POST	`/api/customer/orders`	Order placement
GET	`/api/customer/orders/:id`	Order details

customer.service.ts

- Business logic for customer operations
- Order data validation before creation
- Manages order status transitions

customer.repository.ts

- Database queries for customer data
- Fetching restaurants with availabity status
- Menu items retrival by restaurant
- Create and update orders

Cart Management Structure

```
sequenceDiagram
actor Customer
participant VM as ViewMenuComponent
participant CartS as CartService
participant LS as LocalStorage
participant CP as CartPageComponent

    Customer->>VM: Click "Add to Cart"
    VM->>CartS: addToCart(item, restaurantId)
    CartS->>LS: getCurrentRestaurant()
    alt Different Restaurant
        CartS->>Customer: Confirm clear cart?
        Customer->>CartS: Yes/No
    end
    CartS->>LS: saveCart(updatedCart)
    CartS->>VM: Success
    VM->>Customer: Show snackbar notification
    Customer->>CP: Navigate to cart
    CP->>CartS: getCart()
    CartS->>LS: Retrieve cart data
    LS->>CP: Return cart items
    CP->>Customer: Display cart
```

Distance and Delivery simulations

- Assign fictional coordinates to users and restaurants (e.g., `(x, y)`)
- Compute distances such as Manhattan distance:

$$- Distance = |x1 - x2| + |y1 - y2|$$

- Example:

```
User location: (5, 3)
Restaurant location: (6, 7)
Distance: |5-6| + |3-7| = 1 + 4 = 5 units
Estimated delivery: 5 × 5 = 25 minutes (base) + 15 (prep) = 40 minutes
```

EXTRA TASK

Real-Time Order-Tracking

The order tracking component simulates WebSocket communication to provide customers with live order updates. This creates an engaging, real-time experience only by using WebSocket simulation.

Key features

1. **WebSocket Simulation**
    - Connection simulation: simulates WebSocket handshake and connection duration
    - Connection status: Visual indicator, indicating status when connected, ex ("Live")
    - Automatic reconnection: simulates reconnection attempts if connection disappears
2. **Real-time state updates**
    - Order status automatically progresses based on specific timing intervals
    - Dynamic ETA: delivery time updated based on current status and distance
    - Order status changes:
    1. Placed -> Accepted
    2. Accepted -> Preparing
    3. Preparing -> Ready
    4. Ready -> Delivering
    5. Delivering -> Delivered
3. **Live-Chat System**

- Bi-directional Communication: Restaurant and customer send and receive messages.
- Customer received messages by restaurant staff at each status update. - Customer sends messages to restaurant. Restaurant replies by default with "Thank you for the message. We will get back to you soon"
- Chat history is preserved throughout the order tracking duration
- Constant Popup Notifications: restaurant messages also appear as snackbar notificaton

4. **Location-Based Tracking**

- Grid Coordinate System: Users and Restaurants are assigned `(x,y)`coordinates
- Distance Calculation: Real-time distance updates as order progresses
- Visual Representation: Visual representation of order journey from order placement to delivery

5.**Status Visualization**

- Progress Bard: visual order journey representation
- Live updates: UI updates after each status update

Implementation Details of extra task

WebSocket Service Architecture

```
// Core WebSocket simulation structure
class WebSocketService {
    private connectionStatus = new BehaviorSubject<boolean>(false);
    private messages = new Subject<ChatMessage>();
    private statusUpdates = new Subject<StatusUpdate>();

    connect(orderId: string, userId: string): void {
        // Simulate connection establishment
        setTimeout(() => {
            this.connectionStatus.next(true);
            this.simulateStatusUpdates(orderId);
            this.simulateRestaurantResponse(orderId);
        }, 1000);
    }

    simulateStatusUpdates(orderId: string): void {
        // Timeline-based status progression
        const timeline = [
            { delay: 30000, status: 'accepted' },
            { delay: 90000, status: 'preparing' },
            { delay: 300000, status: 'ready' },
            { delay: 1000, status: 'delivering' },
            { delay: this.calculateDeliveryTime(), status: 'delivered' },
        ];
    }
}
```

Chat Message Structure

```
interface ChatMessage {
    id: string;
    senderId: string;
    senderName: string;
    senderType: 'customer' | 'restaurant';
    message: string;
    timestamp: Date;
    orderId: string;
    showPopup?: boolean; // For important notifications
}
```

Order-Status changes

```
const statusTimeline = {
    placed: { duration: '0-30s', action: 'Order received' },
    accepted: { duration: '30s-2m', action: 'Restaurant confirmed' },
    preparing: { duration: '2-8m', action: 'Food preparation' },
    ready: { duration: '0-1m', action: 'Ready for delivery' },
    delivering: { duration: 'Distance-based', action: 'Out for delivery' },
```

```
    delivered: { duration: 'Final', action: 'Order completed' },
};
```

Challenges

1. Real-time communication simulation- using Websocket **simulation** connection
2. Coordinate updates- ensuring status updates, chat messages and pop up messages are in sync
3. Visual representation

Key Features Summary

1. **Restaurant Filtering**: restaurant search and filtering
2. **Cart Constraints**: restaurant-specific cart
3. **Order Tracking**: real-time updates with visual representation
4. **Payment Section**: payment method options
5. **Feedback Dubmission**: post-delivery feedback submission
6. **Real-Time Communication**: Websocket chat simulation and status updates

Setup instructions

Important: -Node.js -npm -Angular CLI

Backend Setup

1. Navigate to server directory:
    - Type: **cd server**
2. Install dependencies:
    - Type: **npm install**
3. Build and start the server:
    - Type: **npm run build**
    - After type: **npm start**
4. If port 3000 is already in use, terminate the process using it:
    - Type: **netstat -ano | findstr :3000**
    - Then: **taskkill /PID [PID] /F**
5. Restart the server:
    - Type: **npm start**
    - It would show that the Server is usign port 3000

Frontend Setup

1. Open new terminal and navigate to client directory:
    - Type: **cd client**
2. Install dependencies: -Type: **npm install**
3. Configure proxy for API calls: - Make sure proxy.conf.json in client module has this code:

```
  {
  "/api": {
```

```
  "target": "http://localhost:3000",
  "secure": false,
  "changeOrigin": true
  },
  "/uploads": {
  "target": "http://localhost:3000",
  "secure": false,
  "changeOrigin": true
  }
  }
```

- **Important**: Docker usage was unverified due to persistent installation difficulties on the development laptop. This proxy configuration was used instead of the colleage's for easier website check!!

4. Start Angular development server:
    - Type: **ng serve**
5. Access application:
    - Open browser and navigate to: http://localhost:xxxx
    - Use the following test credentials:
        - Email: **customer@freelivery.com**
        - Password: **customer**

Environment Variables

No particular environment variables are required for local development. The application uses default configurations for:

- Database connection (local PostgreSQL)
- Server port (3000)
- API endpoints (localhost)